**Politehnica University of Timișoara**
Faculty of Automation and Computers
Department of Computers and Information Technology

# ANNEX 2

## Bachelor's Thesis — Assignment Sheet

Academic Year 2026

---

| | |
|---|---|
| **Student:** | Tudor-Ciprian TĂMAȘ |
| **Study programme:** | Electronics Telecomunications and Information Technology |
| **Scientific coordinator:** | Sl. Dr. Ing. Valentin MĂRĂNESCU |

---

| | |
|---|---|
| **Thesis title:** | **SHILATE**<br><br>**Software Hardware In Loop Automotive Testing Enviroment** |

---

| | |
|---|---|
| **Keywords:** | reinforcement learning, software-defined vehicle (SDV), simulation-to-real, Eclipse Leda, MQTT, Kuksa databroker, vehicle signal specification (VSS), Proximal Policy Optimisation (PPO), Unity simulation, Gymnasium |

---

### Scope of Work

SHILATE (*Software Hardware In Loop Automotive Testing Environment*) investigates the feasibility of training a reinforcement learning agent to navigate a virtual obstacle course and deploying the resulting policy on an Eclipse Leda software-defined vehicle stack without modifying the training code.

The work encompasses: (i) a Unity 2022 simulation providing a vehicle physics model, a LiDAR-inspired 21-ray perception layer, and a shaped reward function communicated over MQTT; (ii) a Python training pipeline based on Stable-Baselines3 PPO with real-time health telemetry streamed back to the Unity Editor; (iii) an SDV integration layer consisting of an MQTT-to-Kuksa feeder, a Velocitas vehicle safety monitor, and Kanto-managed container deployment on Eclipse Leda; (iv) a theoretical hardware actuator design coupling the agent's throttle output to a 12 V DC pump via a MOSFET driver and Zener flyback suppressor on a Raspberry Pi 4.

---

### Implementation Stages

1. **Unity Simulation Environment**
   Scene construction (`RuntimeSceneBuilder`, circular obstacle course); vehicle physics

with four `WheelCollider` components; 21-ray LiDAR-inspired perception layer (`RaycastSensor.cs`); shaped reward function in `TrainingBridge.cs`.

2. **Custom MQTT Client for Unity**
   Native C# implementation of MQTT 3.1.1 (`MqttClient.cs`) supporting CONNECT, PUBLISH (QoS 0), SUBSCRIBE, PINGREQ/PINGRESP, and DISCONNECT without external dependencies.

3. **Python RL Training Pipeline**
   Gymnasium-compliant `ShilateEnv` wrapper with threading-event synchronisation; Stable-Baselines3 PPO agent with separate actor/critic networks; `HealthCallback` with NaN/Inf loss detection; TensorBoard logging; checkpoint save and resume support.

4. **Unity Training Controller Editor Window**
   `TrainingEditorWindow.cs`: one-click training launch, live metric graphs (episode reward, policy loss, value loss, KL divergence), colour-coded health banner, issue log, and cross-platform Python process management.

5. **SDV Integration — Eclipse Leda Stack**
   `mqtt-kuksa-feeder` container bridging MQTT telemetry to VSS paths via Kuksa gRPC; Velocitas vehicle application monitoring overspeed and RPM redline; Kanto container manifests; `provision-device.sh` deployment automation.

6. **AI Inference Deployment**
   `ai_driver.py` running trained PPO checkpoint in closed-loop inference inside the Leda `leda-controller` container; deterministic policy evaluation with episode auto-reset.

7. **Theoretical Hardware Actuator (Raspberry Pi 4)**
   Circuit design for MOSFET-driven 12 V pump (IRF520N, 1 kΩ gate resistor, Zener/TVS flyback suppressor); Python daemon subscribing to `Vehicle.OBD.ThrottlePositio` via Kuksa and driving GPIO PWM at 1 kHz.

8. **Documentation**
   Bachelor's thesis written in LaTeX following the UPT template; bibliography of 15 references in IEEE format; figures, code listings, and experimental results tables.

---

**Rezumat**

Lucrarea propune *SHILATE* (Software Hardware In Loop Automotive Testing Environment), un mediu de antrenare prin învățare prin întărire pentru vehicule autonome, care integrează o simulare Unity cu un strat de deployment bazat pe Eclipse Leda (platforma de referință Software-Defined Vehicle a Eclipse SDV).

Agentul PPO (Proximal Policy Optimisation) este antrenat într-un circuit circular cu obstacole, utilizând un senzor de tip LiDAR simulat cu 21 de raze și o funcție de recompensă cu modelare explicită a progresului, coliziunilor și etapelor intermediare. Comunicarea între simulare și procesul Python de antrenare se realizează exclusiv prin protocolul MQTT 3.1.1, implementat nativ în C# fără dependențe externe.

Politica antrenată este deployată pe Leda printr-un set de containere gestionate de Kanto: `leda-controller` (inferență), `mqtt-kuksa-feeder` (mapare MQTT → VSS via gRPC) și `velocitas-app` (monitorizare siguranță). Semnalele vehiculului sunt expuse prin intermediul Kuksa Databroker conform specificației VSS (Vehicle Signal Specification), asigurând interoperabilitate cu orice actuator compatibil SDV.

---

| **Scientific Coordinator** | **Student** |
|---|---|
| Sl. Dr. Ing. Valentin MĂRĂNESCU | Tudor-Ciprian TĂMAȘ |
| ..... | ..... |
| (signature) | (signature) |

*Date of issue: July 2026*